Lecture 02 | July, 2026

Search Evaluation

Instructor: Sithvothy Kiv

Slides Credit: Josh Hug

Search Evaluation

- **Search Evaluation**
- Setting up search
- Collecting relevance data
- Search Evaluation Metrics
- Hit Rate
- Mean Reciprocal Rank (MRR)
- Putting it together
- Interpreting the metrics

Search Evaluation

Now that we have ground truth data, we can evaluate how well our search retrieves the correct documents. For each question in our ground truth dataset, we run search.

Then we check whether the results include the correct document.

Setting up search

- Search Evaluation
- **Setting up search**
- Collecting relevance data
- Search Evaluation Metrics
- Hit Rate
- Mean Reciprocal Rank (MRR)
- Putting it together
- Interpreting the metrics

Setting up search

Create a new notebook for search evaluation. We'll use the ground truth CSV from the previous lesson and set up the search index here.
For search evaluation, we only need the search part of the RAG pipeline.

We don't need to call the LLM yet.

Load the ground truth file from the previous notebook:

```
import pandas as pd
df_ground_truth = pd.read_csv("data/ground_truth-new.csv")
ground_truth = df_ground_truth.to_dict(orient="records")
```

Setting up search

Load the documents and build a minsearch index:

```
from ingest import load_faq_data, build_index

documents = load_faq_data()

documents_llm = []

for doc in documents:
    if doc["course"] == "llm":
        documents_llm.append(doc)

documents = documents_llm
index = build_index(documents)
```

Setting up search

Wrap the search call in a function called text_search. The name is deliberate.

Later we'll write vector_search or a hybrid version and run the exact same evaluation on it.

Everything downstream only needs a function that takes a query and returns results, so we can swap one for another.

Setting up search

That mirrors how RAG works: the retrieval step doesn't care which search function sits behind it.

```
def text_search(query):
    boost_dict = {"question": 3.0, "section": 0.5}

    return index.search(
        query,
        num_results=5,
        boost_dict=boost_dict
    )
```

Collecting relevance data

- Search Evaluation
- Setting up search
- **Collecting relevance data**
- Search Evaluation Metrics
- Hit Rate
- Mean Reciprocal Rank (MRR)
- Putting it together
- Interpreting the metrics

Collecting relevance data

Start with one ground truth record and Run search for this question:

```
q = ground_truth[0]

doc_id = q["document"]
results = text_search(query=q["question"])

for d in results:
      print(f'{d["doc_id"]} == {doc_id}: {d["doc_id"] == doc_id}')
```

Collecting relevance data

Then turn this comparison into a relevance list. In this lesson, relevance means whether a retrieved document is the correct document for this question.

```
relevance = []

for d in results:
    relevance.append(int(d["doc_id"] == doc_id))

relevance
```

Collecting relevance data

Put this logic into a function:

```
def compute_relevance_text(q):
    doc_id = q["document"]
    results = text_search(query=q["question"])

    relevance = []
    for d in results:
        relevance.append(int(d["id"] == doc_id))

    return relevance
```

Collecting relevance data

For the first ground truth record, the relevance list is:

```
q = ground_truth[0]
print(q["question"])
compute_relevance_text(q)
# [1, 0, 0, 0, 0]
```

Collecting relevance data

For this question:

```
q = ground_truth[11]
print(q["question"])
compute_relevance_text(q)
# [1, 0, 0, 0, 0]
```

Collecting relevance data

this question:

```
q = ground_truth[50]
print(q["question"])
compute_relevance_text(q)
# [1, 0, 0, 0, 0]
```

Collecting relevance data

Now do the same thing for all ground truth questions:

```
from tqdm.auto import tqdm

def compute_relevance_total_text(ground_truth):
    relevance_total = []

    for q in tqdm(ground_truth):
        relevance = compute_relevance_text(q)
        relevance_total.append(relevance)

     return relevance_total
```

Collecting relevance data

Call it for the first 15 ground truth questions:

```
ground_truth_sample = ground_truth[:15]
relevance_total_text = compute_relevance_total_text(ground_truth_sample)
relevance_total_text
```

Each entry in relevance_total_text is a relevance list. This is enough to check that the function works before we run it for the full dataset.

Collecting relevance data

Next, make the relevance functions generic. We start with text search, but later we may want to evaluate vector search, hybrid search, or another retrieval method.

The relevance logic is the same.

Collecting relevance data

Only the search function changes.

```
def compute_relevance(q, search_function):
    doc_id = q["document"]
    results = search_function(query=q["question"])

    relevance = []
    for d in results:
        relevance.append(int(d["id"] == doc_id))

    return relevance
```

Collecting relevance data

We need to provide it explicitly:

```
def compute_relevance_total(ground_truth, search_function):
    relevance_total = []

    for q in tqdm(ground_truth):
        relevance = compute_relevance(q, search_function)
        relevance_total.append(relevance)

    return relevance_total
```

Collecting relevance data

Use it with text_search on the same sample:

```
relevance_total = compute_relevance_total(ground_truth_sample, text_search)
relevance_total

relevance_total = compute_relevance_total(ground_truth, text_search)
```

Now we can represent search results as relevance lists. In the next lesson, we'll turn these lists into metrics: **Hit Rate** and **Mean Reciprocal Rank (MRR).**

Search Evaluation Metric

- Search Evaluation
- Setting up search
- Collecting relevance data
- **Search Evaluation Metrics**
- Hit Rate
- Mean Reciprocal Rank (MRR)
- Putting it together
- Interpreting the metrics

Search Evaluation Metrics

We have computed relevance lists for search results.

We can turn those lists into metrics.

Hit Rate

- Search Evaluation
- Setting up search
- Collecting relevance data
- Search Evaluation Metrics
- **Hit Rate**
- Mean Reciprocal Rank (MRR)
- Putting it together
- Interpreting the metrics

Hit Rate

Hit Rate (also called Recall@k) measures the fraction of queries where the correct document appears anywhere in the results:

```
example = [
    [1, 0, 0, 0, 0],[0, 1, 0, 0, 0],
    [1, 0, 0, 0, 0],[0, 0, 0, 0, 0],
    [0, 1, 0, 0, 0],[1, 0, 0, 0, 0],
    [1, 0, 0, 0, 0],[1, 0, 0, 0, 0],
    [1, 0, 0, 0, 0],[0, 0, 1, 0, 0],
    [1, 0, 0, 0, 0],[1, 0, 0, 0, 0],
    [1, 0, 0, 0, 0],[1, 0, 0, 0, 0],
    [1, 0, 0, 0, 0],
]
```

Hit Rate

Each line is one query. If a line contains 1, search found the correct document somewhere in the top 5 results. If the line contains only zeros, search did not find the correct document.

In our setup, each query has one correct document, so Hit Rate and Recall@k mean the same thing.

Hit Rate

Let's calculate it:

```
cnt = 0

for line in example:
    if 1 in line:
        cnt = cnt + 1

 cnt
 # There are 14 hits. The example has 15 queries.
```

Hit Rate

The Hit Rate is:

```
cnt / len(example)
# 0.933 This means that search found the correct document for 93.3%
of the queries in this example.
```

Hit Rate

Put the same logic into a function:

```
def hit_rate(relevance):
    cnt = 0

    for line in relevance:
        if 1 in line:
            cnt = cnt + 1

    return cnt / len(relevance)
```

Mean Reciprocal Rank

- Search Evaluation
- Setting up search
- Collecting relevance data
- Search Evaluation Metrics
- Hit Rate
- **Mean Reciprocal Rank (MRR)**
- Putting it together
- Interpreting the metrics

Mean Reciprocal Rank (MRR)

Hit Rate tells us if we found the right document, but not where it was.

MRR also considers the position.

For each query, the score is based on the rank of the first correct document:

- position 1: score is 1.0
- position 2: score is 0.5
- position 3: score is 0.333
- not found: score is 0

In the example, most hits are at the first position. Some hits are lower in the list.

Mean Reciprocal Rank (MRR)

Look at that line:

```
(example[1])
# [0, 1, 0, 0, 0]
```

For this line, the score is 1/2 because the correct document is at position 2.

Mean Reciprocal Rank (MRR)

Let's calculate MRR:

```
total_score = 0.0

for line in example:
    for rank in range(len(line)):
        if line[rank] == 1:
            total_score = total_score + 1 / (rank + 1)
            break

total_score
```

The total score is 12.333333333333334. We use rank + 1 because Python counts positions from zero. The first position should score 1/1, and without the + 1 we'd divide by zero.

Mean Reciprocal Rank (MRR)

Divide it by the number of queries:

```
total_score / len(example)
# 0.822
```

MRR is the average of these scores across all queries. It rewards systems that put the correct document near the top.

Hit Rate is the upper bound for MRR. In practice, MRR is usually smaller because some correct documents are found below the first position.

Mean Reciprocal Rank (MRR)

Put the same logic into a function:

```
def mrr(relevance):
    total_score = 0.0

    for line in relevance:
        for rank in range(len(line)):
            if line[rank] == 1:
                total_score = total_score + 1 / (rank + 1)
                break

    return total_score / len(relevance)

mrr(example)
# 0.822
```

Putting it together

- Search Evaluation
- Setting up search
- Collecting relevance data
- Search Evaluation Metrics
- Hit Rate
- Mean Reciprocal Rank (MRR)
- **Putting it together**
- Interpreting the metrics

Putting it together

Wrap the metrics in a reusable evaluation function:

```
def evaluate(ground_truth, search_function):
    relevance_total = compute_relevance_total(ground_truth, search_function)

    return {
        "hit_rate": hit_rate(relevance_total),
        "mrr": mrr(relevance_total),
    }
```

Putting it together

We can evaluate any search function:

```
evaluate(
    ground_truth,
    text_search
)

# {"hit_rate": 0.899, "mrr": 0.769}
```

Search metrics tell us whether retrieval works. Next, we'll use these metrics to tune the search parameters.

Interpreting the metrics

- Search Evaluation
- Setting up search
- Collecting relevance data
- Search Evaluation Metrics
- Hit Rate
- Mean Reciprocal Rank (MRR)
- Putting it together
- **Interpreting the metrics**

Interpreting the metrics

A few things to keep in mind when reading these numbers:

Our ground truth assumes only one relevant document per query. In practice, other retrieved documents might also be relevant. A 50% hit rate does not mean that half the results are useless.

It means the document we generated the question from did not appear in the top results for half the queries.

Other relevant documents may still be there.

Interpreting the metrics

With synthetic data, the generated questions can be too close to the original FAQ text. This inflates hit rate and MRR. If you see numbers above 95%, treat them with caution and check whether the questions are realistic enough.

Interpreting the metrics

Good thresholds depend on your use case. A 50% hit rate is acceptable for some applications, while others need 90% or higher.

The right number depends on how much the downstream LLM can compensate for imperfect retrieval. It also depends on user tolerance for wrong answers.

Interpreting the metrics

Look at the system holistically.

A high MRR means the relevant document is near the top, which helps the LLM focus on the right context.

A low MRR with a high hit rate means the document is there, but buried under less relevant results.